

Facultad de ingeniería

UNIVERSIDAD NACIONAL DE COLOMBIA



ParCheck
Ingeniería de Software I

Entrega 04

Tutorial

Omar Alejandro Blanco Pineda

Sara Isabel Sepulveda Perez

Juan Esteban Ocampo Vidal

Samuel Mejia Ortiz

Daniel Felipe Orejuela Guzman

13 de mayo de 2026

Tutorial para construcción de aplicación web desde 0

1. Introducción

Este tutorial servirá como herramienta para saber cómo realizar la construcción de una aplicación web desde 0 con el lenguaje de programación python. Para lo anterior, se va a construir una API Rest de Lista de Tareas. Al finalizar, obtendremos una API funcional capaz de crear, leer, actualizar y eliminar tareas, con una base de datos PostgreSQL corriendo en Docker.

Stack elegido: Python + Django + PostgreSQL

¿Por qué este stack?

Python + Django es un stack muy popular en el desarrollo backend. Python es un lenguaje que ofrece una sintaxis clara, Django por su parte es un framework bien estructurado el cual tiene un ORM propio. PostgreSQL es de las bases de datos relacionales de código abierto más avanzadas y fiables.

Django ORM (Object-Relational Mapping)

En este caso el ORM utilizado viene directamente integrado en Django, esta herramienta permite interactuar con la base de datos usando código Python, sin necesidad de realizar Querys. Definiendo los objetos como clases, el ORM traduce las operaciones en querys, facilitando así variedad de tareas como crear, consultar o eliminar los registros de forma sencilla.

Query en SQL:

```
SELECT * FROM tareas WHERE completada = false;
```

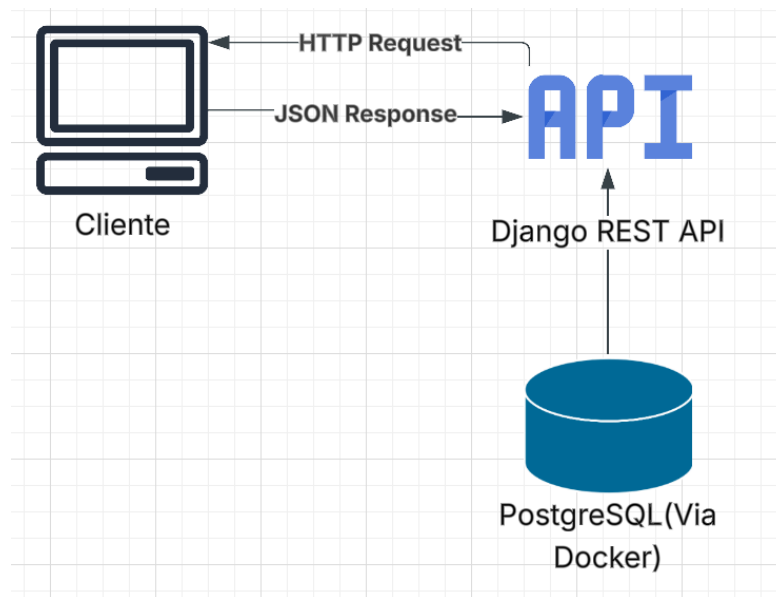
Query desde código en Python:

```
Tarea.objects.filter(completada=False)
```

2. Prerrequisitos (qué necesita tener instalado)

- Python: 3.11+
- Git
- Postman
- Docker desktop

3. Arquitectura del sistema



Flujo de una petición

1. El cliente envía una petición HTTP a la API
2. Django recibe la petición y la enruta a la vista correspondiente
3. La vista usa el ORM para consultar o modificar datos en PostgreSQL
4. Django devuelve una respuesta en formato JSON.

4. Configuración de la base de datos con Docker

Ahora vamos a comenzar por la configuración de la base de datos en PostgreSQL. En este caso no es necesario instalar directamente PostgreSQL por lo que vamos a usar Docker, esto permitirá que cualquier persona que clone el proyecto levante la base de datos fácilmente y de forma estandarizada. Acá solo vamos a crear los archivos necesarios para el levantamiento.

- a. Crear un archivo Dockerfile donde se define como se construye la imagen de Django.

```
FROM python:3.11-slim

WORKDIR /app

# Copiar e instalar dependencias primero
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copiar el resto del proyecto
COPY . .

EXPOSE 8000
```

```
CMD ["python", "app/manage.py", "runserver", "0.0.0.0:8000"]
```

- b. Crear un archivo docker-compose.yml el cual se va a encargar de orquestar los 2 servicios.

```
version: "3.9"

services:
  db:
    image: postgres:15
    container_name: tutorial_db
    restart: always
    environment:
      POSTGRES_DB: todo_db
      POSTGRES_USER: todo_user
      POSTGRES_PASSWORD: password
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d ${DB_NAME}"]
      interval: 10s
      timeout: 5s
      retries: 5

  web:
    build: .
    container_name: tutorial_web
    restart: always
    volumes:
      - ./app
    ports:
      - "8000:8000"
    env_file:
      - .env
    depends_on:
      db:
        condition: service_healthy

volumes:
  postgres_data:
```

El archivo primero indica la versión del formato de Docker Compose. Luego en Services se definen los contenedores que se deben crear. Como muestra el código en el **servicio db**, se define la imagen de base de datos que se va a usar, en este caso postgres:15, también se le asigna nombre al container y definimos dentro la base de datos, el usuario y la contraseña. Por último el puerto que será utilizado.

Por otra parte en el servicio web será el de la app Django. Aquí construye la imagen con el Dockerfile que se creó en el paso anterior. Se define un nombre y puerto para el container, por último crea la dependencia con la base de datos.

- c. Ahora es necesario crear el archivo requirements.txt en la raíz del proyecto el cual va a usar el Dockerfile para instalar dependencias.

```
django==4.2
psycopg2-binary==2.9.9
python-decouple==3.8
```

Como vemos este define las dependencias necesarias para el funcionamiento del proyecto y sus versiones específicas. En este caso incluye Django como framework principal, psycopg2-binary cómo driver que permite la conexión entre Django y PostgreSQL, y python-decouple para gestionar variables de entorno de forma segura, evitando exponer configuraciones sensibles dentro del código.

- d. Por último vamos a crear un archivo .env que almacene las variables de entorno con las configuraciones y credenciales del proyecto.

```
# Base de datos
DB_NAME=todo_db
DB_USER=todo_user
DB_PASSWORD=password
DB_HOST=db
DB_PORT=5432

# Django
SECRET_KEY=django-insecure-cambia-esta-clave-en-producción
DEBUG=True
```

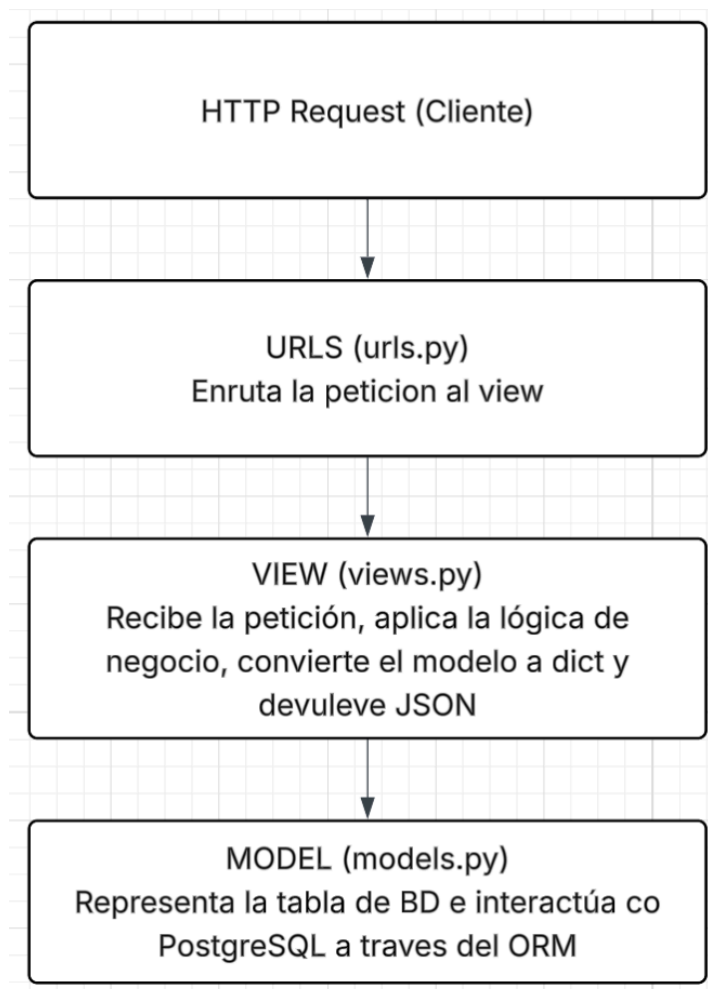
Este define los parámetros de conexión a la base de datos PostgreSQL, como nombre, usuario, contraseña, host y puerto, además de configuraciones propias de Django como la SECRET_KEY, utilizada para la seguridad interna de la aplicación la cual se cambia por la que tenga el proyecto en settings al ser creado, y DEBUG, que activa el modo de desarrollo para mostrar información detallada de errores.

5. Creación del proyecto Django

En esta sección vamos a entender cómo funciona el Framework Django. Este framework utiliza la estructura MTV (Model - Template - View).

- Modelo: hace referencia a la gestión y estructura de la base de datos, aquí entra ORM, permite representar las tablas como clases y realizar consultas, crear, actualizar, eliminar, etc...
- View: es la lógica del sistema, procesa solicitudes y respuestas utilizando los modelos definidos.
- Template: es el “molde visual” de la página, con lo que interactúa el usuario.

De acuerdo con esta estructura, para nuestra aplicación tendríamos:



Para comenzar se realiza la creación del proyecto para lo cual vamos a crear un entorno virtual temporal, ya que los comandos para la creación deben correr en una máquina con Django instalado.

- a. Crear y activar el entorno virtual ejecutando desde un terminal

```
python -m venv venv
.\venv\Scripts\activate
```

- b. Instalar Django dentro del entorno

```
(venv) PS C:\Users\sarai\OneDrive\Escritorio\tutorial-api> pip install django
Collecting django
  Downloading django-6.0.5-py3-none-any.whl.metadata (3.9 kB)
Collecting asgiref<=3.9.1 (from django)
  Using cached asgiref-3.11.1-py3-none-any.whl.metadata (9.3 kB)
Collecting sqlparse<=0.5.0 (from django)
  Using cached sqlparse-0.5.5-py3-none-any.whl.metadata (4.7 kB)
Collecting tzdata (from django)
  Downloading tzdata-2026.2-py2.py3-none-any.whl.metadata (1.4 kB)
Downloading django-6.0.5-py3-none-any.whl (8.4 MB)
 8.4/8.4 MB 25.8 MB/s 0:00:00
Using cached asgiref-3.11.1-py3-none-any.whl (24 kB)
Using cached sqlparse-0.5.5-py3-none-any.whl (46 kB)
Downloading tzdata-2026.2-py2.py3-none-any.whl (349 kB)
Installing collected packages: tzdata, sqlparse, asgiref, django
Successfully installed asgiref-3.11.1 django-6.0.5 sqlparse-0.5.5 tzdata-2026.2

[notice] A new release of pip is available: 25.3 -> 26.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(venv) PS C:\Users\sarai\OneDrive\Escritorio\tutorial-api>
```

- c. Crear el proyecto en la carpeta raíz ejecutando en el terminal en C:\carpeta raíz

```
django-admin startproject config app
```

Con este comando se va a crear dentro de la carpeta, la carpeta app contiene las el proyecto, que es donde se realizan las configuraciones necesarias, con sus respectivos archivos y el manage.py para ejecución.

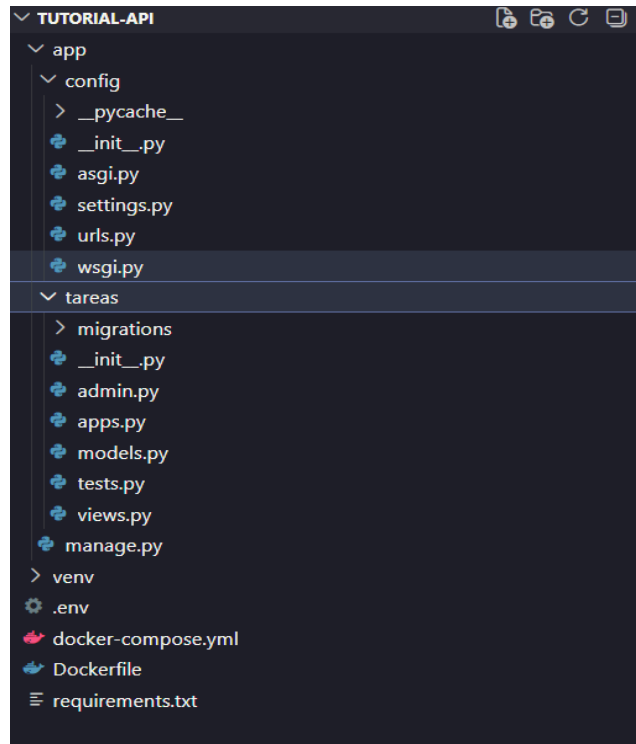
```
▼ TUTORIAL-API
  ▼ app
    ▼ config
      • __init__.py
      • asgi.py
      • settings.py
      • urls.py
      • wsgi.py
      • manage.py
```

- d. Creamos la aplicación tareas en /app donde vamos a tener la lógica de la aplicación, esto lo hacemos ejecutando

```
cd app
```

```
python manage.py startapp tareas
```

Una vez realizado obtenemos así la carpeta raíz final con los archivos docker y django.



- e. Antes de continuar se realizan las respectivas configuraciones del proyecto en /config/settings.py asegurandose de que los datos de apps y bases de datos correspondan a lo que se está realizando y utilizando. Se realizan cambios en:
- INSTALLED APPS: Agregamos nuestra app tareas.

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    ...  
    'tareas'  
]
```

- DATABASES: Acá si es necesario reemplazar la configuración por defecto por la de PostgreSQL como se muestra.

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.postgresql',  
        'NAME': config('DB_NAME'),  
        'USER': config('DB_USER'),  
        'PASSWORD': config('DB_PASSWORD'),  
        'HOST': config('DB_HOST'),  
        'PORT': config('DB_PORT', default='5432'),  
    }
```



```
}  
}
```

- f. Por último desactivamos el entorno virtual temporal, a partir de ahora todo correrá dentro de Docker, por lo que podemos eliminar la carpeta venv/

6. Creación de contenedores en Docker

Una vez creado el proyecto procedemos a levantar la base de datos con los archivos ya configurados, para lo cual ejecutamos en el terminal el comando:

```
docker-compose up --build
```

En la ubicación donde se encuentran todos los archivos del principio. Desde este momento puedes escribir código en tu editor y Django lo recargará automáticamente gracias al volumen volumes: ./app configurado en el docker-compose.yml. Los errores aparecerán directamente en esta terminal.

7. Programación de la aplicación

Procedemos a crear el código en cada una de las capas del sistema.

- a. **Models.py:** Continuamos, realizando la instancia de un modelo, que en este caso sería la tarea, de la siguiente forma:

```

class Tarea(models.Model):

    titulo      = models.CharField(max_length=200)

    descripcion = models.TextField(blank=True, null=True)

    completada  = models.BooleanField(default=False)

    creada_en   = models.DateTimeField(auto_now_add=True)

    class Meta:

        db_table = "tareass"

        ordering = ["-creada_en"]

    def __str__(self):

        return self.titulo

```

Cuando defines una clase en models.py que hereda de django.db.models.Model, Django la interpreta como una entidad de base de datos. Cada atributo de la clase (como título, descripción, completada) representa una columna con su tipo de dato y restricciones. Luego realizamos las migraciones.

```

docker exec -it tutorial_web python app/manage.py makemigrations

docker exec -it tutorial_web python app/manage.py migrate

```

Al ejecutar make migrations, Django analiza esa clase y genera un archivo de migración que describe los cambios en forma de instrucciones Python. Luego, al ejecutar migrate, Django traduce ese archivo a SQL y lo ejecuta contra PostgreSQL, creando físicamente la tabla con todas sus columnas. Es decir, nunca escribes SQL directamente: el ORM de Django toma tu clase Python y se encarga de crear y mantener la estructura de la base de datos por ti.

- b. Views.py:** Definiremos las vistas con funciones nativas de Django. Esto nos permite ver exactamente qué hace cada método HTTP, haciendo el código más transparente y didáctico. Se definieron los siguientes métodos:

Método	Ruta	Funcion
GET	/api/tareas/	Retorna todas las tareas

POST	/api/tareas/	Crea una nueva tarea
GET	/api/tareas/{id}/	Retorna una tarea por ID
PUT	/api/tareas/{id}/	Actualiza una tarea
DELETE	/api/tareas/{id}/	Elimina una tarea

c. URLs: Registro de rutas. Este registro se realiza en dos ubicaciones.

i. `app/tareas/urls.py`

```
from . import views

urlpatterns = [
    path("tareas/", views.lista_tareas, name="lista-tareas"),
    path("tareas/<int:id>/", views.detalle_tarea,
name="detalle-tarea"),
]
```

Cada path mapea una URL a su función correspondiente en `views.py`. El parametro `int:id` captura el ID de la URL y lo pasa automáticamente a la función como argumento.

ii. `app/config/urls.py`

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("api/", include("tareas.urls")),
]
```

Con estos paths todos los endpoints están disponibles bajo el prefijo `/api/`

d. `Admin.py`: Aquí se deben registrar todos los modelos, en este caso solo tarea

```
from django.contrib import admin
from .models import Tarea

admin.site.register(Tarea)
```

8. Pruebas desde Postman

En esta sección probaremos cada uno de los endpoints de la API usando Postman, verificando que la conexión a la base de datos, la instanciación del modelo y las respuestas JSON funcionen correctamente.

Primero debemos ejecutar la aplicación, para lo que hay que ejecutar los contenedores de la siguiente forma la primera vez:

```
docker-compose up --build
```

Para verificar que esta ejecutando correctamente:

```
docker-compose ps
```

Ejecutar normalmente

```
docker-compose up
```

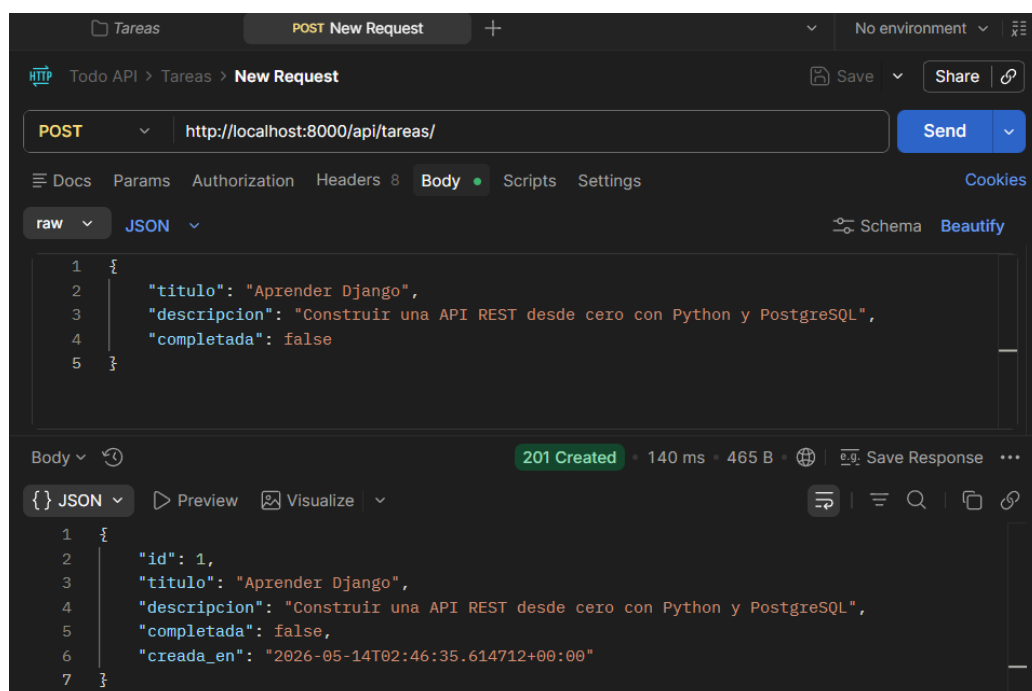
Para comprobar que esté corriendo abre un navegador y entra a <http://localhost:8000/api/> debería aparecer un JSON vacío [].

Antes de comenzar con los HTTP requests, vamos a crear una colección para organizar las pruebas. Para esto dentro de postman vamos a Collections -> New Collection y la nombramos Todo API. Dentro de la colección creamos la carpeta tareas.

Todas las peticiones que vamos a realizar van a ser con la URL base <http://localhost:8000/api/tareas/>

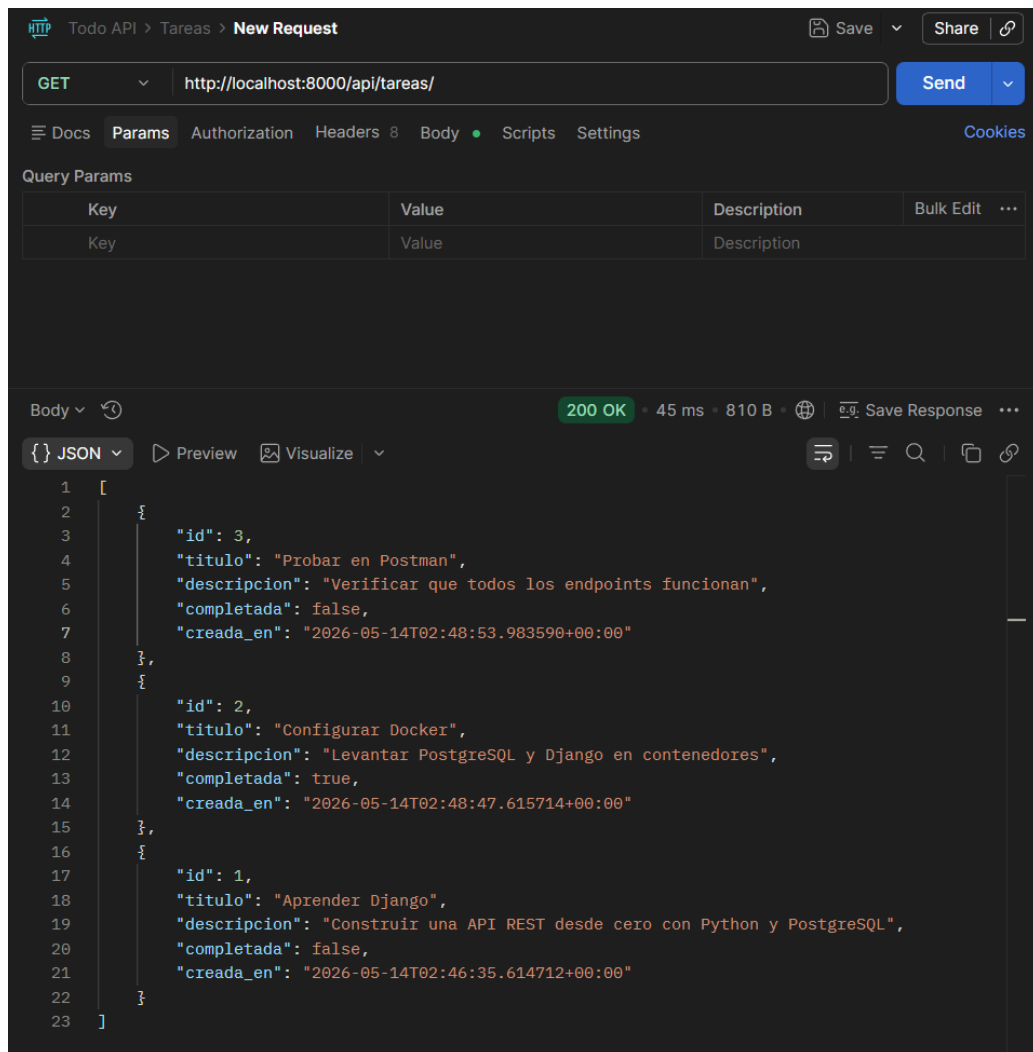
a. Crear una tarea - POST

Esta petición crea una nueva tarea en la base de datos, para ello es necesario pasarle un Body .JSON. Con esta petición se espera una respuesta (status 201 created) y devuelve json con la tarea y su id.



b. Listar todas las tareas - GET

Esta petición retorna todas las tareas almacenadas en la base de datos.



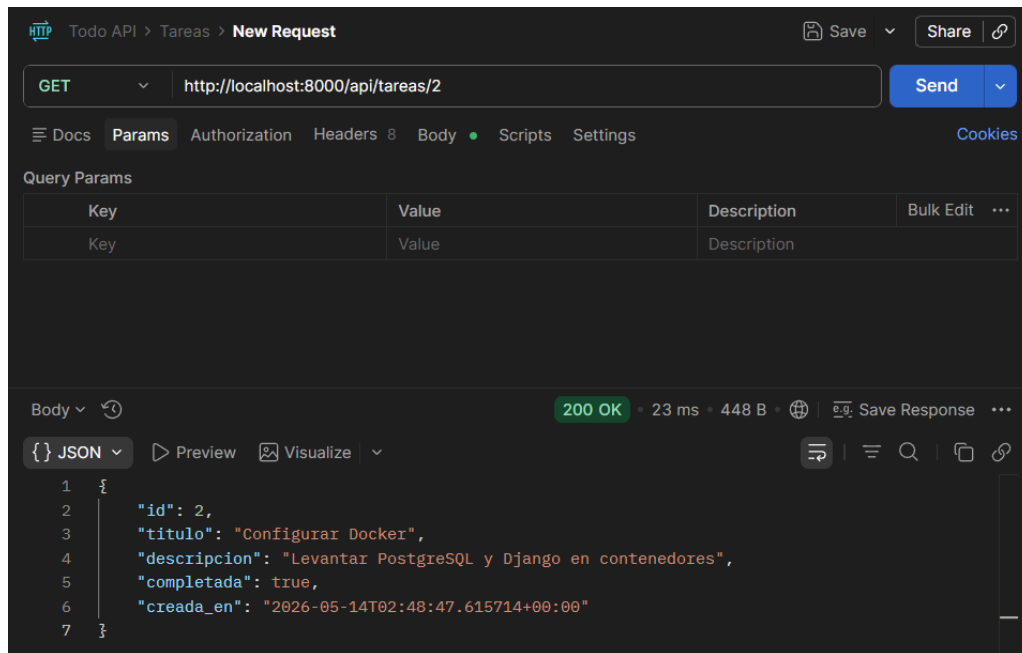
The screenshot shows a REST client interface with a GET request to `http://localhost:8000/api/tareas/`. The response is a 200 OK status with a response time of 45 ms and a body size of 810 B. The response body is a JSON array containing three task objects.

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

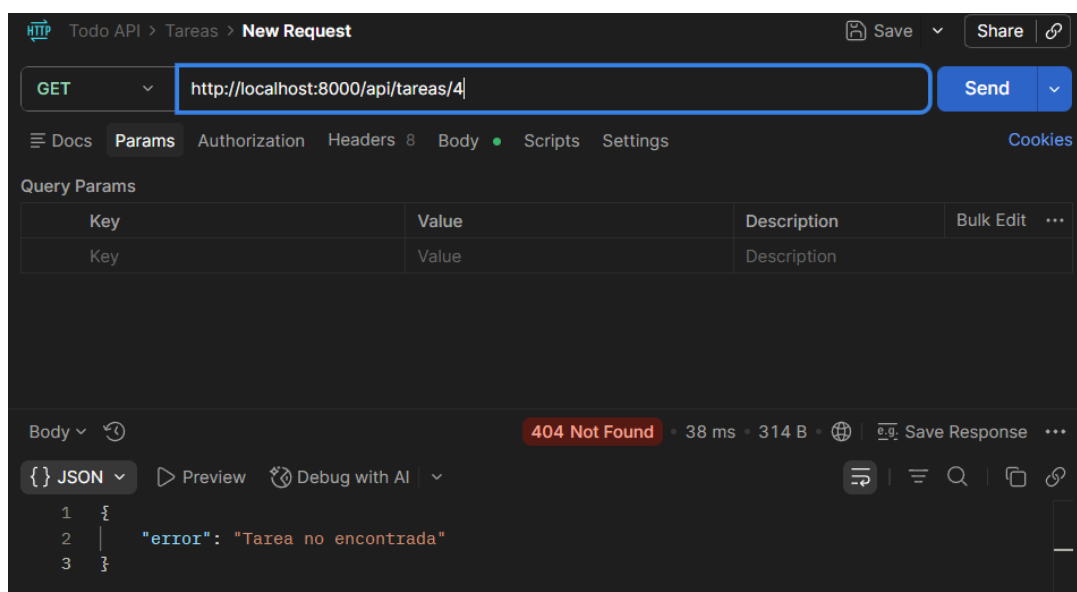
```
[
  {
    "id": 3,
    "titulo": "Probar en Postman",
    "descripcion": "Verificar que todos los endpoints funcionan",
    "completada": false,
    "creada_en": "2026-05-14T02:48:53.983590+00:00"
  },
  {
    "id": 2,
    "titulo": "Configurar Docker",
    "descripcion": "Levantar PostgreSQL y Django en contenedores",
    "completada": true,
    "creada_en": "2026-05-14T02:48:47.615714+00:00"
  },
  {
    "id": 1,
    "titulo": "Aprender Django",
    "descripcion": "Construir una API REST desde cero con Python y PostgreSQL",
    "completada": false,
    "creada_en": "2026-05-14T02:46:35.614712+00:00"
  }
]
```

c. Obtener una tarea por ID - GET

Petición que retorna una tarea de acuerdo con su ID

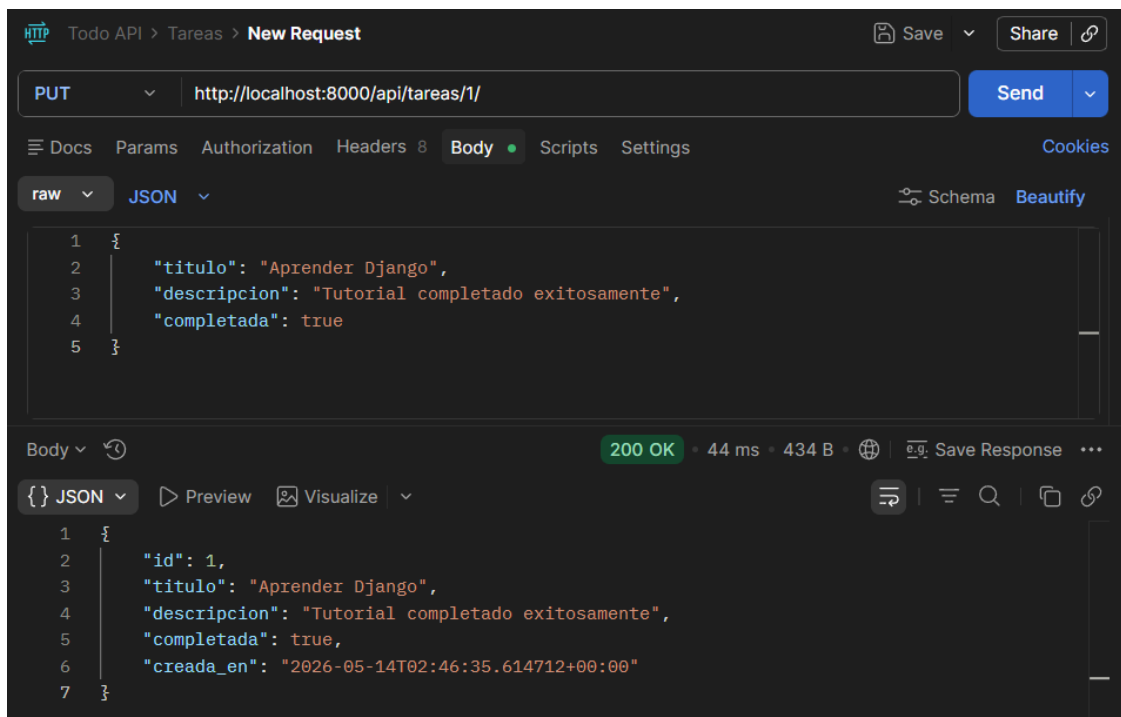


Si el ID no existe devuelve el código 404 not found



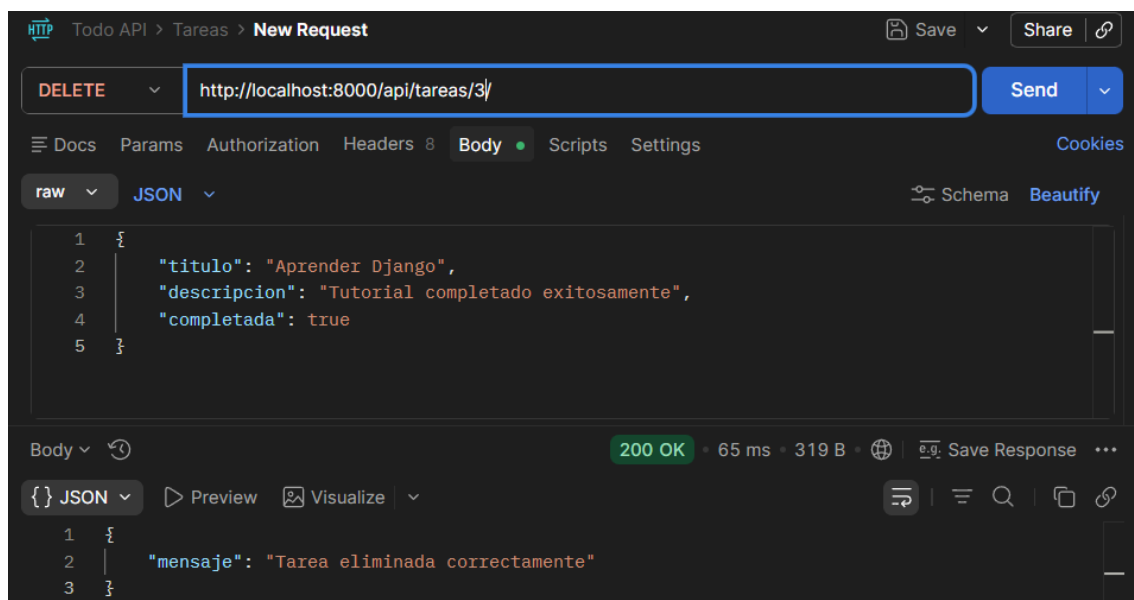
d. Actualizar una tarea - PUT

Esta petición actualiza los datos de una tarea existente, el URL se manda con el ID y se manda en el body un JSON con los cambios realizados.



e. Eliminar una tarea - DELETE

Esta petición elimina una tarea de la base de datos.



Si llegaste hasta aquí, acabas de construir tu primera API REST funcional con Python, Django y PostgreSQL corriendo en Docker. No es un ejercicio teórico: es una aplicación real

con una base de datos real, endpoints que responden peticiones HTTP y un entorno reproducible que cualquier persona puede levantar con un solo comando.